

Practical Week 8

Session 8

Task 1

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.



```
GNU nano 8.7.1 session8_memory.c *
#include <stdio.h>
#include <stdlib.h>

int main() {

    printf("Dynamic Memory Allocation Demo\n");
    printf("=====\n");

    // -----
    // 1. malloc()
    // -----
    printf("\n1. Using malloc()\n");

    int *arr = malloc(5 * sizeof(int));

    if (arr == NULL) {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr[i] = (i + 1) * 10;
    }

    printf("malloc values: ");

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n");

    // -----
    // 2. calloc()
    // -----
    printf("\n2. Using calloc()\n");
```

```
int *zeros = calloc(5, sizeof(int));

if (zeros == NULL) {
    perror("calloc");
    free(arr);
    return 1;
}

printf("calloc values: ");

for (int i = 0; i < 5; i++) {
    printf("%d ", zeros[i]);
}

printf("\n");

// -----
// 3. realloc()
// -----
printf("\n3. Using realloc()\n");

int *temp = realloc(arr, 10 * sizeof(int));

if (temp == NULL) {
    perror("realloc");
    free(arr);
    free(zeros);
    return 1;
}

arr = temp;

for (int i = 5; i < 10; i++) {
    arr[i] = (i + 1) * 10;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^\ **Replace**

^K Cut
^U Paste

^T Execute
^J Justify

```

    }

    printf("After realloc: ");

    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n");

    // -----
    // 4. free()
    // -----
    printf("\n4. Using free()\n");

    free(arr);
    free(zeros);

    printf("Allocated memory has been freed.\n");

    return 0;
}
|
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

```
jella_pragathi@LAPTOP-RQQG555:~$ cd ~/OSSP
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ nano session8_memory.c
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ ^C
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ gcc -Wall -Wextra -g session8_memory.c -o session8_memory
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ ./session8_memory
Dynamic Memory Allocation Demo
=====

1. Using malloc()
malloc values: 10 20 30 40 50

2. Using calloc()
calloc values: 0 0 0 0 0

3. Using realloc()
After realloc: 10 20 30 40 50 60 70 80 90 100

4. Using free()
Allocated memory has been freed.
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ valgrind --version
valgrind-3.26.0
```

```
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ valgrind --leak-check=full ./session8_memory
```

```
==575== Memcheck, a memory error detector
```

```
==575== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
```

```
==575== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
```

```
==575== Command: ./session8_memory
```

```
==575==
```

```
Dynamic Memory Allocation Demo
```

```
=====
```

```
1. Using malloc()
```

```
malloc values: 10 20 30 40 50
```

```
2. Using calloc()
```

```
calloc values: 0 0 0 0 0
```

```
3. Using realloc()
```

```
After realloc: 10 20 30 40 50 60 70 80 90 100
```

```
4. Using free()
```

```
Allocated memory has been freed.
```

```
==575==
```

```
==575== HEAP SUMMARY:
```

```
==575==      in use at exit: 0 bytes in 0 blocks
```

```
==575==    total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
```

```
==575==
```

```
==575== All heap blocks were freed -- no leaks are possible
```

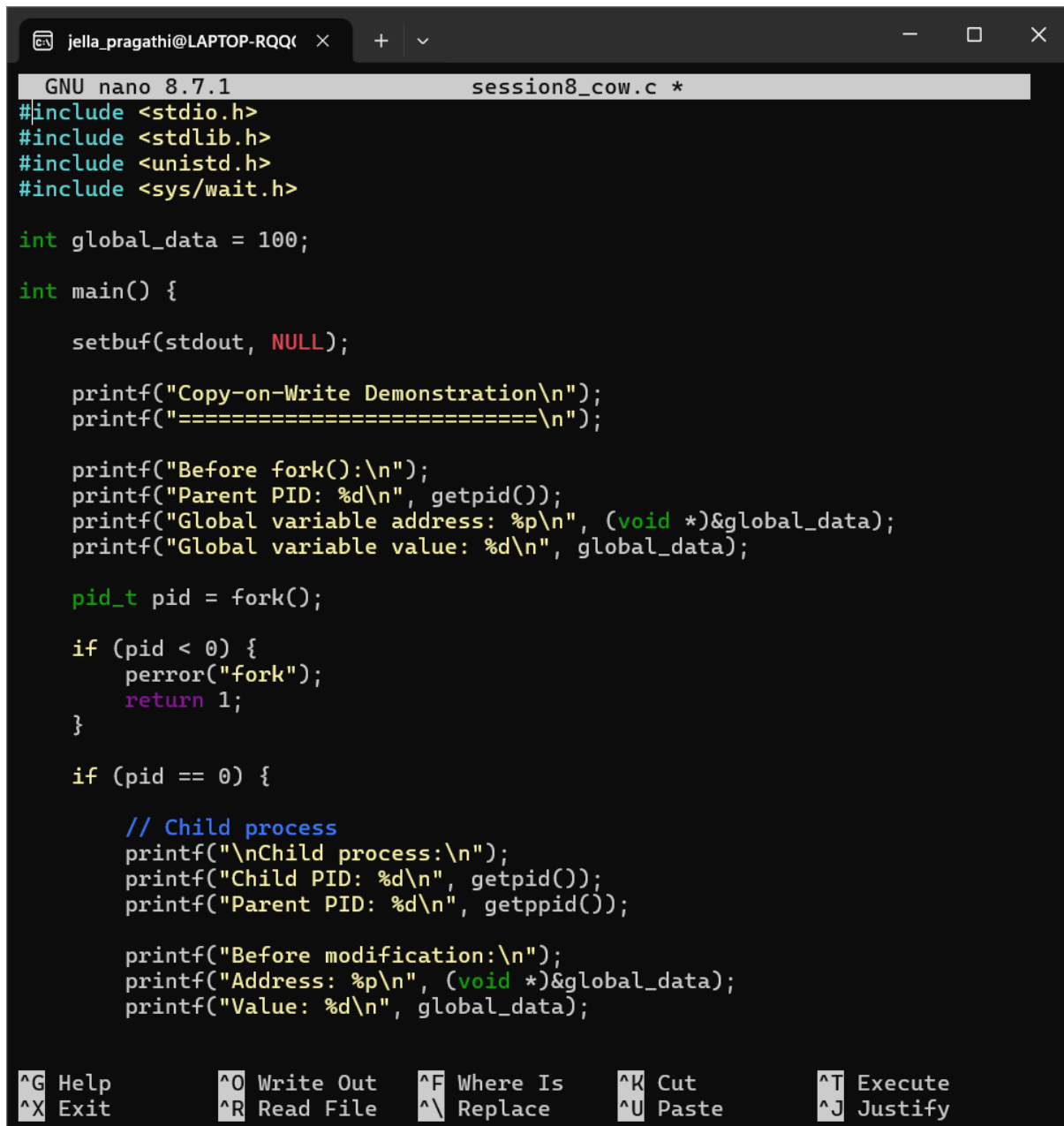
```
==575==
```

```
==575== For lists of detected and suppressed errors, rerun with: -s
```

```
==575== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Task 2

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.



```
GNU nano 8.7.1 session8_cow.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int global_data = 100;

int main() {
    setbuf(stdout, NULL);

    printf("Copy-on-Write Demonstration\n");
    printf("=====\n");

    printf("Before fork():\n");
    printf("Parent PID: %d\n", getpid());
    printf("Global variable address: %p\n", (void *)&global_data);
    printf("Global variable value: %d\n", global_data);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("\nChild process:\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Before modification:\n");
        printf("Address: %p\n", (void *)&global_data);
        printf("Value: %d\n", global_data);
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

        // Modify the variable
        global_data = 200;

        printf("\nAfter child modifies global_data:\n");
        printf("Address: %p\n", (void *)&global_data);
        printf("Value: %d\n", global_data);

        printf("\nChild has its own modified copy.\n");

        exit(0);
    } else {

        // Parent process
        printf("\nParent process:\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent value before child modification: %d\n",
               global_data);

        wait(NULL);

        printf("\nAfter child finishes:\n");
        printf("Parent global_data: %d\n", global_data);
        printf("Parent still has the original value.\n");
    }

    return 0;
}

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^_\} Replace ^{^U} Paste ^{^J} Justify

```

jella_pragathi@LAPTOP-RQQT555:~/OSSP$ cd ~/OSSP
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ nano session8_cow.c
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ gcc -Wall -Wextra -g session8_cow.c -o session8_cow
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ ./session8_cow
Copy-on-Write Demonstration
=====
Before fork():
Parent PID: 668
Global variable address: 0x60b6de69d010
Global variable value: 100

Parent process:
Parent PID: 668
Child PID: 669
Parent value before child modification: 100

Child process:
Child PID: 669
Parent PID: 668
Before modification:
Address: 0x60b6de69d010
Value: 100

After child modifies global_data:
Address: 0x60b6de69d010
Value: 200

Child has its own modified copy.

After child finishes:
Parent global_data: 100
Parent still has the original value.

```